

CS1570 First Month

1. GREEDY ALGORITHMS & MST

Informal definition: makes locally optimal choices at each step with no backtracking. The algorithmic design is usually straightforward but the difficulty lies in proving that it is correct, and indeed, sometimes natural ideas turn out to be wrong, so the correctness proof is key. One correctness proof technique: prove (by induction) that the algorithm never makes a mistake, i.e., the choices made so far can always be augmented into a globally optimal solution. One representative example:

- **MST Problem:** Given undirected, connected, edge-weighted graph $G = (V, E)$ with $w(e) \geq 0$. Find a tree $T \subset E$ spanning all vertices that minimizes $w(T) = \sum_{e \in T} w(e)$. Spanning trees have $n - 1$ edges.
- **Kruskal’s Algorithm:** Builds an acyclic forest. Starts with $T \leftarrow \emptyset$, sorts edges by ascending $w(e)$, and adds e_i if $T \cup \{e_i\}$ remains acyclic.
- **Correctness Highlights:** Kruskal’s connectivity is proven by contradiction via the lightest crossing edge. Optimality is proven by induction: the current choice of edges can be extended to an optimal solution (up to modifying the optimal solution by trading one edge for another).

Advanced example of Greedy: **Huffman coding.**

Prefix-Free Code: No codeword is a prefix of another, ensuring unique decodability without ambiguity.

Tree Representation: Represented as a full binary tree with letters sitting exclusively at the leaves. Path from root dictates the codeword; leaf depth equals codeword length.

Goal: Minimize $\sum_{x \in S} \text{freq}(x) \cdot \text{depth}(x)$. Property:

High-frequency items must sit closer to the root.

Huffman’s Bottom-Up Algorithm:

- **Base Case:** If $|S| = 2$, create a root with two children.
- **Recursive Step:** If $|S| > 2$, find the two least frequent letters x and y . Merge them into a virtual parent node z with $\text{freq}(z) = \text{freq}(x) + \text{freq}(y)$. Recursively solve for $S' = S \setminus \{x, y\} \cup \{z\}$. Reconstruct final tree by expanding z with branches 0 (for x) and 1 (for y).

2. DIVIDE-AND-CONQUER

Divides a problem into a few subproblems, recurses to solved each subproblem, then combines the solutions. Usually the subproblems are on disjoint parts of the input. For those types of algorithms computing the runtime can be a significant part of the difficulty. Here are two well-known and one less common examples:

- **Merge Sort:** Sort halves recursively.
 $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \mathcal{O}(n \log n)$.
- **Binary Search:** Halve search space.
 $T(n) = T(n/2) + \mathcal{O}(1) \Rightarrow \mathcal{O}(\log n)$.
- **Strassen (1969):** Uses additions/subtractions to compute matrix multiplication using only 7 recursive products instead of 8. Recurrence:
 $T(n) = 7T(n/2) + \mathcal{O}(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \mathcal{O}(n^{2.81})$.

When you are lucky the recurrence for the runtime is covered by the **Master Theorem:** For $T(n) = aT(n/b) + f(n)$ where $a \geq 1, b > 1, f(n) > 0$. Compare $f(n)$ to $n^{\log_b a}$:

1. If $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ for some $\epsilon > 0 \Rightarrow T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for $\epsilon > 0$ and $af(n/b) \leq \gamma f(n)$ ($\gamma < 1$) $\Rightarrow T(n) = \Theta(f(n))$.

When you are unlucky, you have to draw the recursion tree or develop the recurrence to ”unfold” it, guess the runtime solution, and prove it by induction. One interesting example is **Deterministic Selection (BFPR 1972):** Finds the element of rank k in an unsorted array in linear worst-case time.

Algorithm: (1) Divide A into $n/5$ groups of 5. Sort each group ($\Theta(1)$) and extract medians into array M . (2) Recursively find median of medians: $m \leftarrow \text{Select}(M, |M|/2)$. (3) Compute absolute rank r of m in A ($\Theta(n)$). (4) If $k = r$ return m ; if $k < r$ recurse on elements smaller than m with rank k ; if $k > r$ recurse on elements greater than m with rank $k - r$.
Analysis: At most $7n/10$ elements remain for the final recursion. Recurrence: $T(n) \leq T(n/5) + T(7n/10) + cn$. Since $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, work decreases geometrically: $T(n) \leq 10cn = \mathcal{O}(n)$.

An advanced example of divide-and-conquer is **Polynomial Multiplication via FFT:**

Representations: Coefficient vector $(a_0, \dots, a_{n-1}) \rightarrow \mathcal{O}(n^2)$ product via convolution. Point-Value form $\{(x_i, A(x_i))\}_{i=0}^{n-1}$ at n sample points $\rightarrow \mathcal{O}(n)$ pointwise product.

Divide-and-Conquer Framework: Split polynomial $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$. Selecting complex roots of unity ($\omega^n = 1$) as evaluation points guarantees a collapsing structure under squaring (x^2). Evaluation (Vandermonde matrix) and interpolation can then be completed in $\mathcal{O}(n \log n)$ time using the Fast Fourier Transform.

3. DYNAMIC PROGRAMMING (DP)

Solves problems by combining solutions to overlapping subproblems stored in a table to prevent redundant work. Requires three ingredients: a recurrence relation, a table, and order in which to fill the table such that when you compute an entry, you have already finished computing the other entries that it needs to look at in its calculation. Here is an example with a 2-dimensional table:

- **Matrix Chain Multiplication:** Multiplying $A_{i \times j}$ by $B_{j \times k}$ costs $\mathcal{O}(i \cdot j \cdot k)$. In what order should we multiply $A_1 \dots A_n$?
Subproblem: Let $C(i, j)$ be the minimum cost to compute the chain product $A_i \dots A_j$. The last multiplication splits the chain at some point l ($i \leq l < j$).
Recurrence: $C(i, i) = 0$, and for $j > i$:
 $C(i, j) = \min_{i \leq l < j} \{C(i, l) + C(l + 1, j) + x_{i-1}x_lx_j\}$.
Table & Order: Upper-triangular table $C[1 \dots n, 1 \dots n]$ filled diagonally by increasing chain length $j - i$. Target is $C[1, n]$. Runtime: $\mathcal{O}(n^3)$.

Another example: **Edit distance**

Definitions: $ed(x, y) = \min$ length of an edit script (insertion, deletion, substitution) transforming x into y . An alignment lines up strings using gap symbols (–). Total cost $al(x, y)$ sums column penalties (0 if identical, 1 if mismatched/gap). Fundamental equivalence: $ed(x, y) = al(x, y)$.

Standard DP Formulation: Let

$D(i, j) = al(x[1 \dots i], y[1 \dots j])$.
Recurrence: $D(0, 0) = 0, D(i, 0) = i, D(0, j) = j$. For $i, j \geq 1$:
If $x_i = y_j, D(i, j) = D(i - 1, j - 1)$. Otherwise:
 $D(i, j) = 1 + \min\{D(i - 1, j), D(i, j - 1), D(i - 1, j - 1)\}$.
Complexity: Table of size $(m + 1) \times (n + 1)$ filled row by row. Runtime: $\mathcal{O}(mn)$, Space: $\mathcal{O}(mn)$.

Linear Space Optimization: Only the current and previous rows are needed to evaluate costs, reducing space to $\mathcal{O}(m + n)$ via two rolling rows.

Advanced example: **Linear Space Edit Sequence**

(Hirschberg’s Algorithm): To reconstruct the actual edit sequence in $\mathcal{O}(m + n)$ space, use divide-and-conquer.

Core Idea: The optimal edit path crosses the middle row $i = m/2$ at some column h , splitting the grid into two independent subproblems.

Algorithm: Compute $h(i, j)$ (the middle-row column index crossed by the path to (i, j)) concurrently with the rolling-row DP.

- For $i < m/2$: $h(i, j)$ is undefined.
- For $i = m/2$: $h(i, j) = j$.
- For $i > m/2$: $h(i, j)$ inherits the tracking value from the minimization neighbor in $D(i, j)$.

Extract $h = h(m, n)$ at the final row. Recursively run $\text{EDITPATH}(A[1 \dots m/2], B[1 \dots h])$ and $\text{EDITPATH}(A[m/2+1 \dots m], B[h+1 \dots n])$, then concatenate.

Analysis: Total time

$T(m, n) = cmn + T(m/2, h) + T(m/2, n - h) \leq 2cmn = \mathcal{O}(mn)$.

Total space is successfully bounded to $\mathcal{O}(m + n)$.

Complexity Summary Table

Problem / Algorithm	Time	Space
Kruskal’s Algorithm (Sorting)	$\mathcal{O}(m \log m)$	$\mathcal{O}(m)$
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Binary Search	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
Strassen’s Algorithm	$\mathcal{O}(n^{2.81})$	$\mathcal{O}(n^2)$
Linear Selection (BFPR)	$\mathcal{O}(n)$	$\mathcal{O}(n)$
FFT / Polynomial Product	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Matrix Chain Parenthesiza- tion	$\mathcal{O}(n^3)$	$\mathcal{O}(n^2)$
Edit Distance (Standard DP)	$\mathcal{O}(mn)$	$\mathcal{O}(mn)$
Edit Distance (Rolling Rows)	$\mathcal{O}(mn)$	$\mathcal{O}(m + n)$
Edit Sequence (Traceback)	$\mathcal{O}(mn)$	$\mathcal{O}(mn)$
Edit Sequence (Hirschberg)	$\mathcal{O}(mn)$	$\mathcal{O}(m + n)$